

../assets/ruet_logo.png

Rajshahi University of Engineering & Technology, Rajshahi
Department of Computer Science & Engineering

Project Design Report

CSE 3206 – Software Engineering Sessional
Lab 2: Software Process Models, Requirement Analysis & MVP Development

QuickPOS – Super Shop POS System (MVP)

Group	Name	Section	Contribution
08	Tanzirul Islam	A (2nd 30)	Server & Frontend base, Git setup, Report
08	Shuvo	A (2nd 30)	Dashboard feature
08	Ruan	A (2nd 30)	Product category + Receipt feature

Submitted to: Course Teacher, Dept. of CSE, RUET

Repository: <https://github.com/TanzirulIslam22/QuickPOS>

Date: September 21, 2026

1 Team Information

Group	Member	Section	Primary Contribution
08	Tanzirul Islam	A (2nd 30)	MVP base (server + frontend), GitHub setup, report
08	Shuvo	A (2nd 30)	Dashboard feature (branch <code>shuvo/dashboard</code>)
08	Ruan	A (2nd 30)	Category + receipt feature (branch <code>ruan/category-receipt</code>)

2 Project Title

QuickPOS – A Super Shop Point of Sale (POS) System (Minimum Viable Product).

3 Problem Statement

A small super shop records sales manually on paper and tracks stock in notebooks. This causes three problems: (1) billing is slow and mistakes happen, (2) the owner does not know which products are in stock or running out, and (3) there is no record of daily sales or how much cash the cashier gave back to a customer. The owner needs a simple, cheap system where a cashier can log in, add items, see the total, enter the cash the customer paid, and get the change to return – while the stock and every sale are saved automatically.

4 Project Objectives

1. Provide a fast and simple billing interface for cashiers.
2. Maintain product information (name, price, stock, category).
3. Automatically reduce stock when a product is sold.
4. Calculate the total bill and the change to return from cash paid.
5. Store and display a complete sales history.
6. Protect the system with a simple staff login.
7. Provide demo data so the system can be demonstrated immediately.

5 Stakeholder Analysis

- **Shop Owner** – needs sales reports, stock control, and a low-cost tool.
- **Cashier / Staff** – needs a fast, easy-to-learn billing screen.
- **Customer** – expects an accurate total and correct change.
- **System Administrator** – manages products and user access.

All stakeholders identified above expect the system to be fast, reliable, and simple to use.

6 Functional Requirements (minimum 10)

1. The system shall allow a staff member to log in using a username and password (`admin/1234`).
2. The system shall list all products with name, price, stock and category.
3. The system shall allow adding a new product.
4. The system shall allow editing an existing product.
5. The system shall allow deleting a product.

6. The system shall allow adding products to a cart with quantity.
7. The system shall calculate the total amount of the cart.
8. The system shall accept the cash amount paid by the customer.
9. The system shall calculate the change to return (**change = paid - total**).
10. The system shall reject a sale when the paid amount is less than the total.
11. The system shall reduce product stock on every sale.
12. The system shall store each completed sale with its items, total, paid and change.
13. The system shall show a list of past sales (history).
14. The system shall seed demo products for testing.
15. The system shall show a dashboard summary of total and today's sales/revenue. (Increment 2)

7 Non-functional Requirements (minimum 8)

1. **Performance:** a checkout must complete in under 2 seconds.
2. **Usability:** the UI must be simple, with no training manual required.
3. **Availability:** runs locally on the shop computer.
4. **Maintainability:** code separated into modular frontend and backend layers.
5. **Portability:** works on any OS through a web browser.
6. **Security:** basic authentication controls access.
7. **Reliability:** every confirmed sale is stored in the database.
8. **Scalability:** supports up to 10,000 products without slowdown.
9. **Integrity:** stock count can never go negative.

8 User Stories / Use Cases (minimum 5)

- US1:** *As a cashier*, I want to log in quickly so that only staff can use the system.
- US2:** *As a cashier*, I want to add products to a cart and see the running total so that billing is fast and accurate.
- US3:** *As a cashier*, I want to enter how much cash the customer gave and see the change, so I give back the correct amount.
- US4:** *As a shop owner*, I want product stock to decrease automatically after each sale, so I never sell out-of-stock items.
- US5:** *As a shop owner*, I want to see past sales and totals, so I can track daily revenue.

9 Assumptions and Constraints

- Assumption: single shop, single billing terminal, no multi-user sessions.
- Assumption: offline operation is acceptable (no cloud needed).
- Assumption: a fixed admin account is enough for the MVP.
- Constraint: three-member team, short lab time, MVP scope only.
- Constraint: first milestone of a semester-long project; must stay modular.

10 Selected Software Process Model

The team selected the **Incremental Development** model. (This matches the model suggested for *Super Shop POS System* in the assignment list.)

The project is delivered as a series of working increments:

Increment 1 – Core Billing: login, product list, cart & checkout with total.

Increment 2 – Stock & History: automatic stock deduction and sales history.

Increment 3 – Cash Handling: paid amount and change calculation.

Increment 4 – Dashboard & UX: summary dashboard, product category, receipt.

Every increment ends with a working, demonstrable product.

11 Justification of Process Model

- **MVP is the goal:** the lab explicitly asks for a small but functional prototype (Phase 3). Incremental starts from a thin but working core and grows feature by feature.
- **Requirements are clear but decomposable:** the POS requirements are well understood, so increments can be planned; the system naturally splits into small modules (products, billing, stock, history), each fitting one increment.
- **Early feedback:** each increment is demoable, so the “client” (course teacher) can give feedback – exactly how the cash-handling feature was added in a later increment without touching the core.
- **Team work with Git:** three members each own one feature branch/increment and merge by Pull Requests, which maps perfectly onto the module’s Phase 4 workflow (main + member1/2/3 feature branches).
- **Semester-long project:** lab instructions say each lab builds on the previous one; incremental delivery supports future labs on design patterns and testing.
- **Low risk and cheap to change:** if one increment fails, only that slice is affected, not the whole system.

12 Comparison with Alternative Models

Why Waterfall is less suitable. Waterfall requires all requirements to be frozen before development and delivers the whole system in one go at the end. The lab demands an early MVP and feedback, and our project evolved (for example, cash-paid/change was added after the first demo). In Waterfall, such a change means going back to the start of a previous phase, which is slow and costly for a small team.

Why Spiral is less suitable. Spiral is designed for large, high-risk, expensive systems where each loop does deep risk analysis and prototype evaluation (e.g., voting systems or airport baggage tracking). A POS system for a small shop is low-risk with stable technology (React + Express + MongoDB). Spiral would add heavy process overhead with no real benefit for a 3-member MVP.

Why Agile Scrum is less suitable. Scrum needs a Product Owner, sprints, daily stand-ups and continuous client meetings. For a short, well-scoped academic MVP with three students and fixed requirements, this ceremony is overkill. Incremental gives the same benefit (working software delivered bit by bit) with far less process weight. The remaining models (Prototype, RAD, XP, Kanban) are meant for unclear UI requirements or different project sizes, and are not a better fit here.

13 MVP Design Overview

13.1 High-level architecture

Client-server architecture:

- **Frontend** (React + Vite + Tailwind): pages Login, Products, Billing (POS), Sales; simple tab navigation in `App.jsx`.
- **Backend** (Node.js + Express): REST API under `/api/products` and `/api/sales`.
- **Database** (MongoDB): collections `Product` and `Sale`.

13.2 Core modules

1. **Authentication** – `Login.jsx` (admin / 1234, stored in `localStorage`).
2. **Product management** – `Products.jsx` + `productRoutes.js` (CRUD).
3. **Billing / POS** – `POS.jsx` + `saleRoutes.js`: cart, total, paid amount, change, stock deduction, sale storage.
4. **Sales history** – `Sales.jsx` (latest 50 bills with total/paid/change).

13.3 Cash handling logic (added as an increment)

The checkout endpoint receives `{items, paidAmount}`, computes the total, validates `paidAmount >= total`, and saves `change = paidAmount - total` into the `Sale` document. The UI shows a live *Change to Return* box and disables the checkout button until enough cash is entered.

Field	Meaning
<code>items</code>	sold items (productId, name, price, qty)
<code>total</code>	bill total
<code>paidAmount</code>	cash given by customer
<code>change</code>	cash to return (<code>paidAmount - total</code>)

Table 1: Sale document fields (incremental addition)

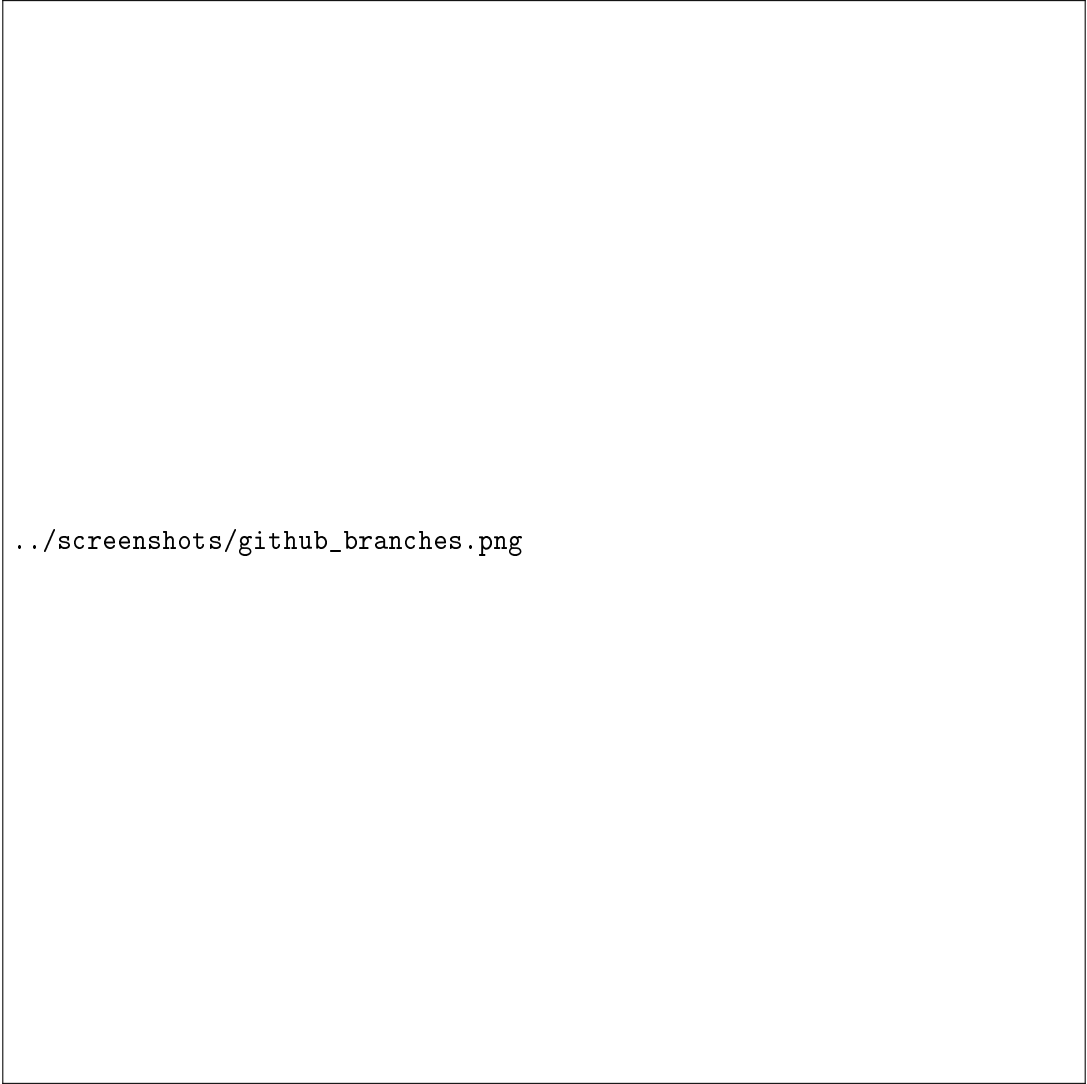
14 GitHub Collaboration Evidence

Member	Branch	Contribution
Tanzirul	<code>tanzirul/mvp-base</code>	repo setup, MVP code, README, docs
Shuvo	<code>shuvo/dashboard</code>	dashboard + summary API
Ruan	<code>ruan/category-receipt</code>	category input + receipt modal

Workflow used: `init/clone` → feature branch → commit (meaningful messages) → push → Pull Request → teammate review (e.g., “LGTM”) → merge to `main`. Full instructions in `docs/TEAM_WORKFLOW.md` and checklist in `docs/TODO.md`.

15 Challenges Encountered

- First-time Git branching and Pull Requests; merge conflicts were avoided by always running `git pull origin main` before making a new branch.
- Validating cash payment on both client and server so insufficient cash can never create a sale.



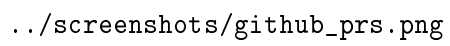
`./screenshots/github_branches.png`

Figure 1: Branches on GitHub

- Keeping the frontend and backend in sync (field names like `paidAmount/change`).
- Working from different machines; solved with a common `.env.example`.

16 Conclusion

QuickPOS demonstrates a functional MVP for a super-shop POS system built with the **Incremental** development model. The team selected and justified the model, documented requirement analysis, and delivered working software in increments through professional Git/GitHub collaboration with feature branches and pull requests. As the first milestone of a semester-long project, the code is modular and ready for future extensions such as design patterns and automated testing.



../screenshots/github_prs.png

Figure 2: Three Pull Requests merged into main